

dash.el

The dash.el list library provides functions to iterate over lists. The *-each* function, for example, takes a list and a function, and applies the function to every element in the list. In the following example, a reverse of the input list is created by doubling each element's value.

```
(-each list function) ;; Syntax

(let (s) (-each '(1 2 3) (lambda (item)
  (setq s (cons (* item 2) s)))) s)
(6 4 2)
```

Another function API from the dash.el library is the *-each-while* function, which takes three arguments – a list, a predicate and a function. The function is applied to every element in the list that satisfies the predicate. For example:

```
(-each-while list predicate function)
;; Syntax

(defun even? (num) (= 0 (% num 2)))
(let (s) (--each-while '(1 2 3 4) (< it
  3) (!cons it s)) s)
(2 1)
```

The *-each-r* function takes a list and a function, and applies the function on every item in the list in the reverse order. An example is given below:

```
(-each-r list function) ;; Syntax

(let (s) (-each-r '(1 2 3) (lambda
  (item) (setq s (cons (* item 2) s))))
s)

(2 4 6)
```

If you would like to use a predicate function with *-each-r*, you can use the *-each-r-while* function as illustrated below:

```
(-each-r-while list predicate function)
;; Syntax
```

```
(let (s) (-each-r-while '(1 2 3 4 5 6)
  'even? (lambda (item) (!cons item s)))
s)
(6)
```

The *-dotimes* function will repeatedly call a function from 0 to the input number, minus 1.

```
(-dotimes number function) ;; Syntax

(let (s) (-dotimes 3 (lambda (n) (!cons
  n s))) s)
(2 1 0)
```

You can explore more of the iterative functions available in dash.el under the *Side-effects* section in the GitHub source repository available at <https://github.com/magnars/dash.el>.

loop.el

We shall now explore the constructs available in the loop.el library. The *loop-while* construct executes the body of the loop while the condition is true. In the following example, the sum of the numbers from 0 to 5 is computed.

```
(require 'loop)

;; loop-while
loop-while (condition body...) ;;
Syntax

(let ((x 0)
      (sum 0))
  ;; sum of 0..5
  (loop-while (< x 5)
    (setq sum (+ sum x))
    (setq x (1+ x)))
  sum)
10
```

If you want to evaluate the body at least once before checking the condition, you can use the *loop-do-while* construct. In the following example, the value of x is incremented by one and then the condition is satisfied for the loop execution.

The Best IT Professionals Work With Open Source.
DO YOU?



16th Edition

OPEN
SOURCE INDIA

17th & 18th October, 2019
NIMHANS Bengaluru

Asia's #1 Conference
On Open Source

Mark the Dates on Your Calendar.

Registrations Open at
www.OpenSourceIndia.in

Want to Engage with India's Best Software
Developers & IT Managers? Exhibit at OSI.

www.OpenSourceIndia.in
osi@efy.in | +91-98111 55335 |